ARITMÉTICA NO SHELL

*O monitor rodava a cada cinco minutos e nunca alertou. O disco chegou a **97%** e o script continuou calado — sem erro de `df`, sem erro de permissão. O `percentual` valia **zero**, e valia zero desde o primeiro dia. A aritmética do shell é só de **inteiros**, e ela não avisa onde a fração se perde: não há erro, não há `NaN` — há um número redondo e errado.*

063
GUIAS LINUX

ONDE A FRAÇÃO SE PERDE

`usado=41943040 · total=52428800 · ocupação real: 80%`

`$(( usado / total * 100 ))`	**0**	A divisão acontece **primeiro** e **0,8 vira 0**. Zero vezes 100 continua zero: o alerta nunca dispara, com o disco vazio ou cheio.
`$(( usado * 100 / total ))`	**80**	**Multiplique antes de dividir.** Correção de uma posição de caractere — e a que resolve quase todos os casos.
`$(( usado * 10000 / total ))`	**8000**	**Ponto fixo:** trabalhe em centésimos e formate na hora de imprimir — `printf '%d.%02d%%' $((c/100)) $((c%100))` → **80.00%**.
`$(( (a + b/2) / b ))`	**≈**	**Arredonda** em vez de truncar (soma metade do divisor). E `$(( (a + b - 1) / b ))` é o **teto** — quantos blocos, páginas ou partes de tamanho fixo cabem.

A divisão inteira **trunca em direção a zero**: `$(( -7 / 2 ))` vale **−3**, não −4, e `$(( -7 % 2 ))` vale **−1** — é a regra do C, não a matemática modular. Toda expressão com `/` merece a pergunta: *o intermediário cabe em inteiro?* Inteiros são de **64 bits com sinal** e estouram **em silêncio**, dando a volta para negativo. Quando a fração é o próprio resultado — média, desvio, raiz —, pague o processo: `printf '%.2f'` (builtin, sem processo) · `awk 'BEGIN{printf "%.1f", s/n}'` (sempre instalado) · `bc -l` (**sem `-l` ou `scale`, o `bc` trunca igual ao shell**, e não vem instalado por padrão). **Nunca uma vez por item dentro do laço:** acumule inteiros e divida uma vez no fim.

ZERO À ESQUERDA É OCTAL · E O COMANDO QUE DEVOLVE CÓDIGO

mes=$(date +%m)	$((mes))	$((10#mes))
07	7	7
08	value too great	8
09	value too great	9
010	8	10
12	12	12

O prefixo `0` significa **octal**, como em C — e isso atinge tudo que o sistema formata com zero à esquerda: `date +%m`, `+%d`, `+%H`, `+%M`, número de série, código de agência, octeto de IP. O sintoma clássico é um script que roda o ano inteiro e **quebra em agosto e setembro**; o `010` é pior — nunca falha e sempre calcula errado. Defesa: `mes=$(( 10#$(date +%m) ))`, preferível a `${mes#0}`, que transformaria `00` em vazio. As outras bases: `$(( 0x1F ))` =31 e `$(( 2#1011 ))` =11.

`(( x = 5 + 3 ))`
COMANDO. Avalia, atribui e não imprime nada — o que ele devolve é um **código de saída**.

`echo $(( 5 + 3 ))`
EXPANSÃO. Avalia e produz o texto `8`. É POSIX; o `(( ))`, o `let`, o `**` e o `$RANDOM` são do Bash.

A inversão: `(( 1 ))` → `$?=0` (sucesso) e `(( 0 ))` → `$?=1` (falha). Em C zero é falso; no shell zero é sucesso, e o `(( ))` faz a tradução. **Consequência sob `set -e`:** `(( contador++ ))` com o contador em zero devolve o valor *anterior* ao incremento — zero — e **aborta o script** numa linha que parecia inofensiva. As três defesas: **`(( ++contador ))`** · **`(( contador++ )) || :`** · **`contador=$(( contador + 1 ))`**.

Dentro de qualquer contexto aritmético — `(( ))`, `$(( ))`, índice de array — as variáveis **dispensam o cifrão** e não sofrem divisão em palavras; **variável não definida vale zero**, conveniente para contador e traiçoeiro para configuração ausente. Os operadores são os de C: `+ - * / % **`, `++ --`, `<< >> & | ^ ~`, `== ≠ < >`, `&& || !`, o ternário `? :` e as atribuições compostas. **Evite os herdados:** `let` exige citar a expressão, e `expr` é um **programa externo** — um *fork* por conta, com escape obrigatório no `*`. `declare -i` avalia toda atribuição àquela variável, mas engole texto virando **0 em silêncio**.

`$RANDOM` **não é criptograficamente seguro.** É um gerador pseudoaleatório simples, com semente previsível e período curto — senha, *token*, chave e nome de arquivo temporário que precise ser imprevisível **não saem dele**. Nomear temporário com `$RANDOM` num diretório compartilhado abre **corrida de link simbólico**: quem prevê o nome cria o link antes de você. As fontes corretas: `token=$(head -c 32 /dev/urandom | base64)`, `openssl rand`, e `tmp=$(mktemp)` para temporários — nunca invente o nome. E há um viés estatístico à parte: `RANDOM % 100` favorece os valores baixos, porque 32768 não é múltiplo de 100; para sorteio uniforme, `shuf -i 0-99 -n 1`.

RESUMO RÁPIDO

- ✓ **A aritmética do shell é só de inteiros e a fração se perde sem aviso.** `$(( a / b * 100 ))` dá zero sempre que `a < b` — **multiplique antes de dividir**.
- ✓ **`(( ))` é comando e devolve código; `$(( ))` é expansão e produz texto.** No `(( ))`, valor zero vira código de **falha**: `(( c++ ))` com `c=0` aborta um script sob `set -e`. Use `(( ++c ))`.
- ✓ **Em contexto aritmético as variáveis dispensam o cifrão** e a não definida vale zero.
- ✓ **Ponto fixo resolve a maioria dos relatórios:** escale por 100 ou 1000, calcule em inteiro e formate com `printf '%d.%02d'`. Guarde `(a + b/2)/b` para arredondar e `(a + b - 1)/b` para o teto.
- ✓ **Zero à esquerda é octal:** `$(( 08 ))` é erro e `$(( 010 ))` é 8. Force a base com `$(( 10#$valor ))`.
- ✓ **Inteiros de 64 bits com sinal estouram em silêncio**, dando a volta para negativo.
- ✓ **Ponto flutuante de verdade:** `awk` **ou** `bc -l` — e nunca uma vez por item dentro do laço.
- ✓ **`$RANDOM` não gera senha nem nome de temporário.** Use `/dev/urandom`, `openssl rand` ou `mktemp`.

NO KALI

Cálculo de intervalo entre requisições é aritmética inteira, e o erro tem consequência. Um script que calcula a pausa como `$(( 1 / taxa ))` segundos dá **zero** para qualquer taxa maior que 1 — o laço passa a bater no alvo sem pausa nenhuma, o que é indistinguível de uma negação de serviço acidental e derruba justamente o serviço que você precisava analisar. Trabalhe em milissegundos (`pausa_ms=$(( 1000 / taxa ))`) ou use o `--max-rate` da própria ferramenta. **Aritmética sobre octetos de IP quebra em zero à esquerda:** alvos escritos como `010.010.010.005` — formato que aparece em planilha de escopo e em exportação de inventário — fazem `$(( octeto ))` ler `010` como **8**, e o resultado é um endereço **fora do escopo autorizado** calculado a partir de um dentro dele. Varredura fora do escopo contratado é crime pela Lei 12.737/2012: aplique `10#` em todo octeto antes de qualquer conta e valide o endereço reconstruído contra a faixa. **Contador sob `set -e` derruba script de varredura no primeiro alvo** — é justamente quando o contador está em zero que o `(( total++ ))` devolve código 1, e o script morre antes de fazer qualquer coisa útil. **Somar tamanhos de captura passa de 32 bits rápido:** um `.pcap` de algumas horas já tem centenas de milhões de bytes, e multiplicado por um fator de escala para média em kbit/s o intermediário cresce depressa — o teto de 64 bits aguenta, mas confira o resultado, porque estouro não emite aviso, só troca o sinal. E **o `bc` não vem instalado no Kali** (`sudo apt install bc`); o `awk` está sempre lá, o que faz dele a escolha segura para ponto flutuante em script que vai rodar em máquina alheia.